The for loop, in practice

— and the project begins · TC1037 · Lecture 08

Dr. Leopoldo Rodríguez Salazar

Friday 16 October 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Your backpack holds four books

You are carrying four. You need a fifth.

Which one goes?

Algorithms to Live By, ch. 4 — *Caching*

Least Recently Used

Evict the item you touched **longest ago**.

- Every cache you own does it: processor, browser, phone.
- Why: the best predictor of *needing something soon* is *having needed it recently*.
- Belady, 1966: the truly optimal rule needs to know the future. LRU is the best you can do without it.
- The book: your desk, your closet, your home screen — last used on top; the bottom is what to drop.

How do you *find* the least recently used of four? Look at each, remember the oldest so far, replace it when you see an older one. **A loop with a best-so-far inside.**

Your turn — 3 minutes, on paper

`INPUT:` four books, each with the date you last opened it `OUTPUT:` the one to remove

Numbered steps: the loop, the memory for the oldest so far, the comparison.

Underline one line
The line that says **how many times** the loop runs.

Every exercise today has a line you can underline like that.

Exercise 1 — `average_of(n)` · read exactly n things

```
def average_of(n):
    total = 0
    for i in range(n):
        prompt = "Number " + str(i + 1) +
            ": "
        value = float(input(prompt))
        total = total + value
    return total / n
```

The loop counts the inputs. No asked, no step line.

`str(i + 1)`: humans count from 1; + joins text.

`average_of(0)`: the loop runs zero times — fine — and then `0 / 0`. The loop is safe; the line *after* it is not. `main` refuses `n <= 0`.

Exercise 2 — `max_of(n)` · keep the best one

```
def max_of(n):
    best = float(input("Number 1: "))
    for i in range(2, n + 1):
        prompt = "Number " + str(i) + ": "
        value = float(input(prompt))
        if value > best:
            best = value
    return best
```

The first value *is* the best so far — by definition.

`max_of(1)` with 7 → 7.0: zero turns, first value is the answer.

`best = 0` is wrong: try −4 −9 −2.

The start of a best-so-far is the first candidate, not an invented one.

Exercise 3 — a loop inside a loop

```
def table(n):
    for i in range(1, 11):
        print(n, "x", i, "=", n
              * i)

def all_tables(up_to):
    for n in range(1, up_to +
        1):
        table(n)
        print()
```

The inner loop runs **completely** for every turn of the outer one.

`all_tables(3)`: $3 \times 10 = 30$ lines.
`all_tables(12)`: say it before running.

Turns multiply. Three loops deep is n^3; $n = 1000$ is a billion. When a program is slow, look here first.

The inner loop is *a function call*: `all_tables` does not know how `table` works. Your project's `main()` will call `grafica1(tablero)` the same way.

Exercise 4 — draw

```
def bars(n):
    for i in range(1, n + 1):
        print("*" * i)

def pyramid(n):
    for i in range(1, n + 1):
        print(" " * (n - i) + "*"
            * (2 * i - 1))
```

`"*" * 3` is `"***"` — QE1's A2 (`"5" + "3"`), now useful.

`pyramid(3)`: trace row 1 — 2 spaces, 1 star.

```
  *
 ***
*****
```

`bars(0)`: nothing, no error.

Exercise 5 — how many darts?

`spread(trials, 10)` = largest − smallest of ten runs of `estimate_pi(trials)`

100 darts	spread ≈ 0.7
10 000 darts	≈ 0.05
1 000 000 darts	≈ 0.005

100× more work buys 10× less spread.

The square root. That is Monte Carlo's price — and why it needs a fast computer.

Write the four · from 18:20 · algorithm on paper first

File	Function(s)	Shell tests to write down
`A_Matricula_x1.py`	`average_of(n)`; `main` refuses `n <= 0`	10 20 30 → 20.0; `main` with 0 → message
`A_Matricula_x2.py`	`max_of(n)`, same rule	4 9 2 → 9.0; `(1)` with 7 → 7.0; −4 −9 −2 →
`A_Matricula_x3.py`	`table(n)`, `all_tables(up_to)`	`all_tables(3)`: 30 lines — count first
`A_Matricula_x4.py`	`bars(n)`, `pyramid(n)`	`bars(0)` → nothing; `pyramid(3)` exactly

Paper first for 1 and 2. Photos + files → **Week 4 evidence box**. Finished? Make `pyramid` hollow.

Situación problema — released today

1. **Teams of two, today.** Names on the board before you leave.
2. **Deliverable 1 — the dataset — Friday 6 November.** 30–50 rows, up to 10 columns, half text half numbers, a real source. **Start looking this week.**
3. **The rubric is published** — site, *How it is graded*. Program · report · a two-minute oral explanation of *your* code.
4. **You wrote the skeleton an hour ago:** the lab's `menu()` + `main()` with a `while` until *Salir*. That *is* the final program's `main`. The functions it calls are weeks 7–9.

lrodriguez15.github.io/situacion-problema.html · Canvas → Modules → *Situación problema*

Before Tuesday

1. **Homework 5** — `is_perfect`, `longest_collatz`, `main`. **Sunday 18 Oct, 23:59.**
2. **Lab 4 file** — `D_matricula.py` + photo → *Laboratory 4*. Same Sunday.
3. **Your team and a dataset candidate.** Early is the whole game with data.

Quick exam 4: Tuesday 20 October — the `for` loop.

Midterm: Friday 23 October — weeks 1 to 4. *Self-study: prepare for your midterm* is the map.

Turns multiply.

Say the number before you run it.